



Travel Booking Website
CS 465 Project Software Design Document
Version 1.0

Table of Contents

CS 465 Project Software Design Document	1
Table of Contents	2
Document Revision History	2
Instructions	2
Executive Summary	3
Design Constraints	3
System Architecture View	3
Component Diagram	3
Sequence Diagram	5
Class Diagram	5
API Endpoints	7
The User Interface	7

Document Revision History

Version	Date	Author	Comments
0.1	01/25/26	Kyle Simonetti	Added Executive Summary, Design Constraints, and Component Diagram.
0.2	02/08/26	Kyle Simonetti	Added Sequence Diagram, Class Diagram, and API Endpoints.
1.0	02/22/26	Kyle Simonetti	Revision

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

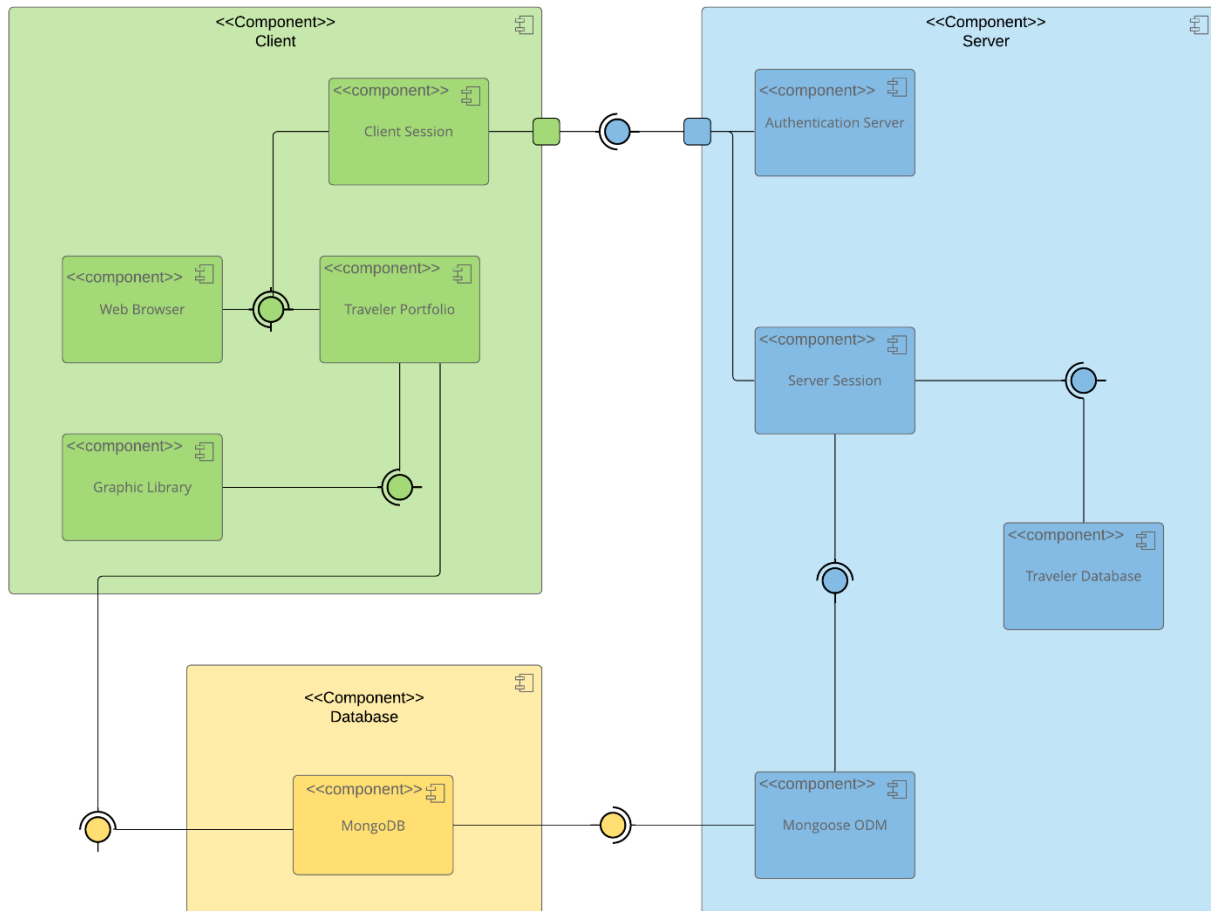
To meet the client's software requirements, appropriate architecture of the web application will be needed. Customers will be able to create an account on the website, browse travel packages by location and price point, book travel packages, allow customers to regularly visit the website to see their itineraries, and an admin-only site where admins can maintain a customer base, available trip packages, and pricing for each item and package. To satisfy all requirements provided, I will be making use of the MEAN stack consisting of MongoDB for the database, Express for the backend framework, Angular for the frontend framework, and Node.js for the server environment to create a scalable web application for the client. Three main web pages will be created, consisting of the homepage, travel page, and admin page. The homepage will contain navigation to other pages, login and registration for new and existing users, and the corporation logo. The travel page will contain a browsable list of travel packages with details such as the resort name and stars, location, and pricing information, and will be able to add a trip if wanted. Lastly, the admin page will allow for an admin to edit or add packages. The database created using MongoDB will maintain travel packages and customer information such as their login and packages booked.

Design Constraints

There are a few design constraints to consider when developing this web application. For one, because we are dealing with customer information and payment methods, we will have to ensure that authentication and authorization are handled correctly and are secure to protect sensitive information. Input sanitization and data validation will be needed to prevent common web attacks such as injection attacks. Secondly, the server will need to handle high traffic while maintaining fast speeds for customers and admins alike. Performance and scalability will be crucial to allow customers to easily book travel packages without slowdown or frustrating load times. Queries will have to be optimized to prevent slowdown for customers. Lastly, building the admin page as a SPA will make the initial load of it slower, which could affect how fast administrators can update travel packages or maintain customer base. Security will also need to be considered with SPA's, as without proper precautions, malicious scripts could be injected which may expose sensitive customer information.

System Architecture View

Component Diagram



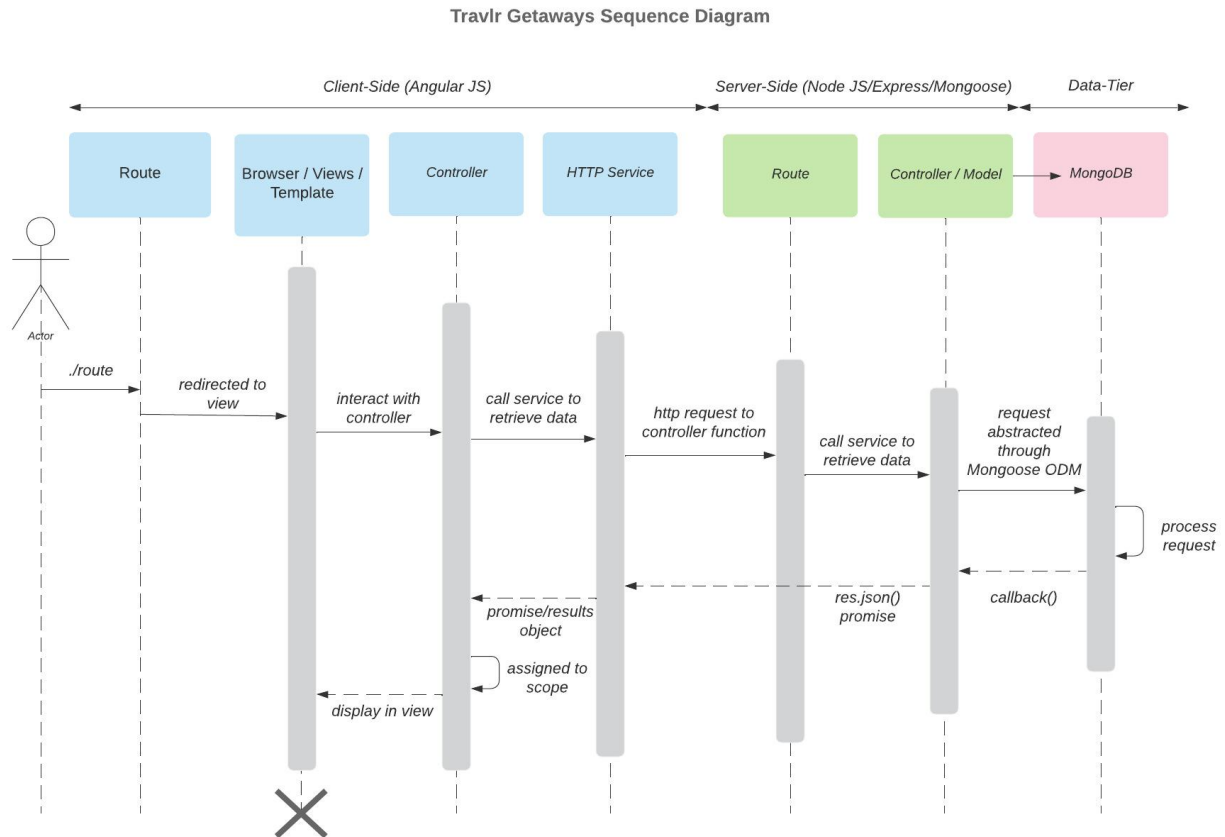
A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version](#).

The overall system architecture consists of three layers: the client, server, and database layers. The client layer is the front-end and built with Angular, that being what the user sees and interacts with, and contains the web browser component, which consists of the graphical interface that the customer will interact with such as adding trips. This browser interfaces with the client session to manage user sessions such as logging in, and the traveler portfolio which manages user-specific information such as their itineraries. The client session interacts with the authentication server to login and register users. Lastly, the travel portfolio interfaces with the graphic library which renders visuals for the user to see and interact with which are dynamically updated.

The server layer contains the back-end logic and built with Node.js and Express, and contains the authentication server which handles the login and validation of the user, ensuring that the username and password entered by the user is valid, and interfaces with the client session to ensure secure logins. The server session component maintains the server-side session states, and interfaces with the traveler database to retrieve and update traveler data, and Mongoose ODM to read or write data from MongoDB. The traveler database component stores travel package information, like location and cost. Lastly, the Mongoose ODM component helps MongoDB and the server session and traveler database to communicate with each other, translating JavaScript objects to documents readable by MongoDB.

The database layer contains one component, MongoDB. Which stores all the application's data.

Sequence Diagram



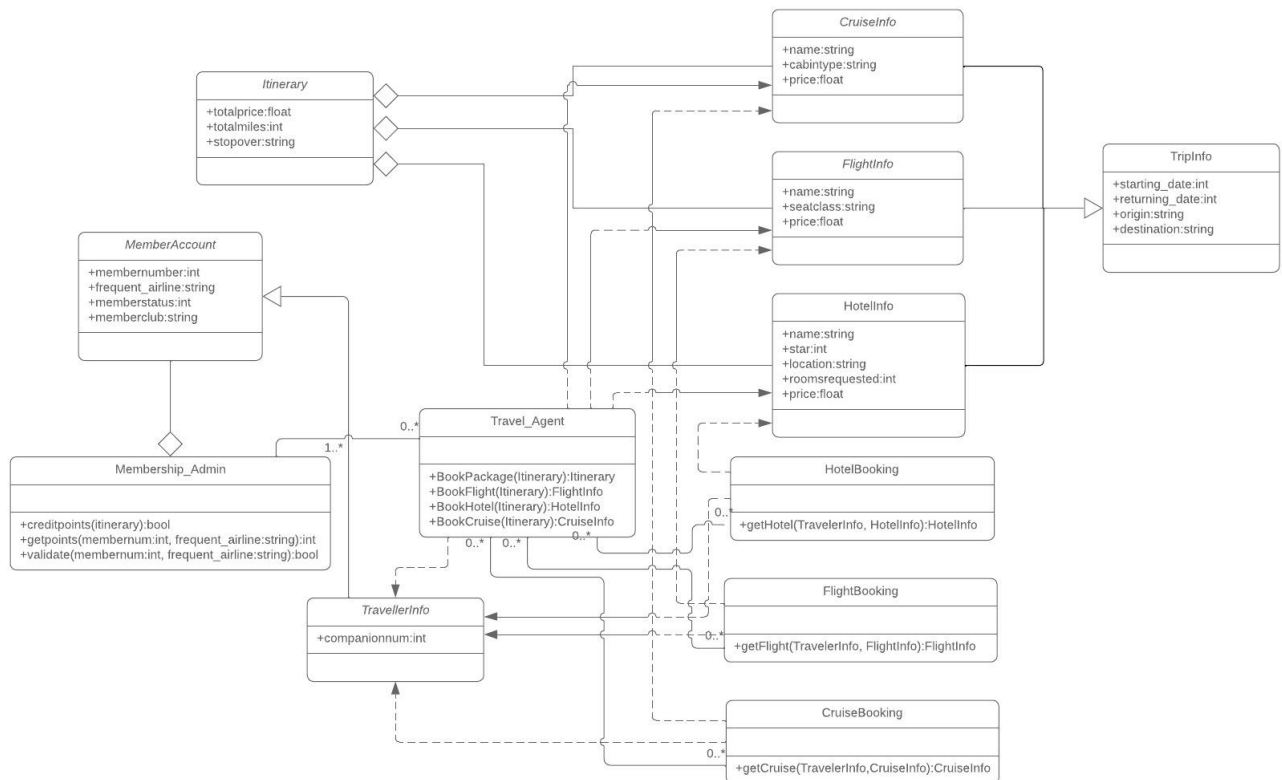
The actor, which is the user of the web application, starts by entering an address into the browser, which the Angular route then redirects the user to a view or template based on what was entered in the browser. With the new page loaded, the user will interact with the page, which is then handled by the controller. Once the user interacts with the page, such as clicking on a link, displaying travel trips to the user, or adding new travel packages by the admin, the controller will handle this user's input, determine what is needed to be retrieved by the server, and calls the HTTP service to collect the data. The HTTP service will then connect with the server to retrieve the requested data, which starts with the Express route redirecting the request to the controller / model.

The server-side controller then uses the model Mongoose ODM to communicate with MongoDB by abstracting database interactions to create a query to get or update the data in MongoDB, sending the result back to the controller.

The controller then sends a JSON response back through the Express route to the HTTP service and sends the result back to the client-side controller. The user's view is then updated based on the data returned by the controller, such as an updated list of all trips being shown to the user, the user being logged into the web application, or an admin seeing their updated trips list after deleting or adding a trip.

Class Diagram

Travlr Getaways Class Diagram



In the web application, there are a total of 12 JavaScript classes, that being the Itinerary, MemberAccount, Membership_Admin, TravellerInfo, Travel_Agent, CruiseInfo, FlightInfo, HotelInfo, HotelBooking, FlightBooking, CruiseBooking, and TripInfo. The Itinerary class has three attributes, a float containing the total price of a trip, an integer of the total miles a trip has, and a string containing what stopovers a trip may have. This class aggregates FlightInfo, HotelInfo, and CruiseInfo. These three info classes contain a string containing their name, an integer of how much they will cost, and specific info such as a string containing what class a user is flying, or an integer of how many stars a hotel has. The TripInfo class is the superclass of the three other info classes, meaning they all inherit the attributes of the TripInfo class, that being a start date, return date, origin, and destination.

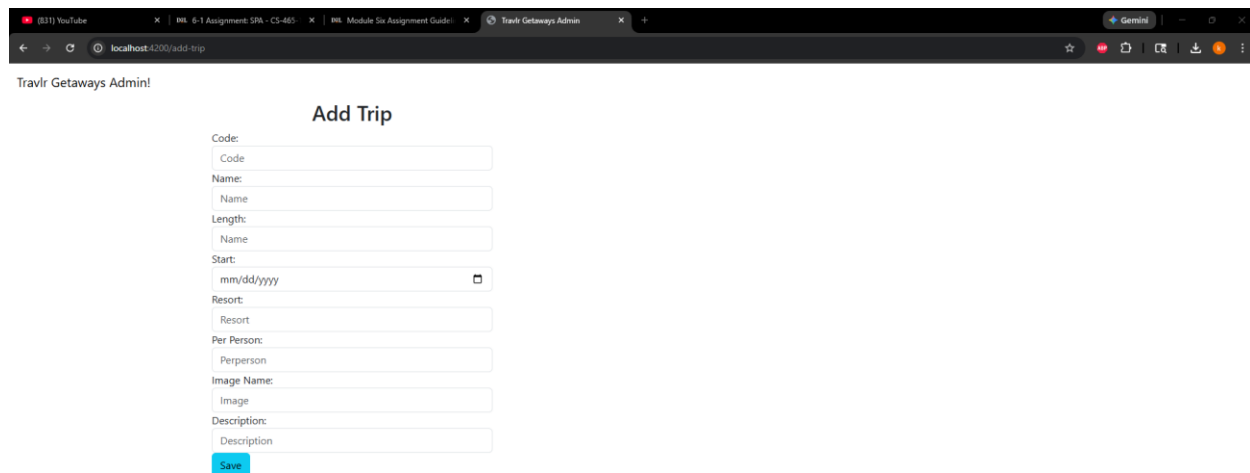
The three booking classes, HotelBooking, FlightBooking, and CruiseBooking contain a get method which takes TravellerInfo and its own info class to get the info on its respective trip type. The TravellerInfo class has one attribute being their companion number, and inherits 4 attributes from the MemberAccount class, that being a member number, a most frequent airline, a member status, and a member club, while also having a one-to-many relationship. The TravelAgent class has four methods that allow them to book trips and packages. The TravelAgent class has a zero-to-many relationship with the booking classes that goes both ways, as a user can have zero bookings or multiple.

Lastly, the `Membership_Admin` class aggregates the `MemberAccount` class, and contains three methods for the administrator to manage `TravelAgents`, which they can have zero-to-many. These methods allow the admin to credit points, get points of a user, and validate a membership of a user.

API Endpoints

Method	Purpose	URL	Notes
GET	Retrieve a single trip by the trip code	/api/trip.:tripcode	Returns a single trip from the database whose trip code matches the trip code provided
GET	Retrieve all trips	/api/trips	Return all trips in the database unfiltered
POST	Login user	/api/login	Authenticates a user using JWT authentication
POST	Register user	/api/register	Registers a user to the database
POST	Add a trip	/api/trips	Adds a trip to the database
PUT	Update a trip	/api/trips	Updates a trip

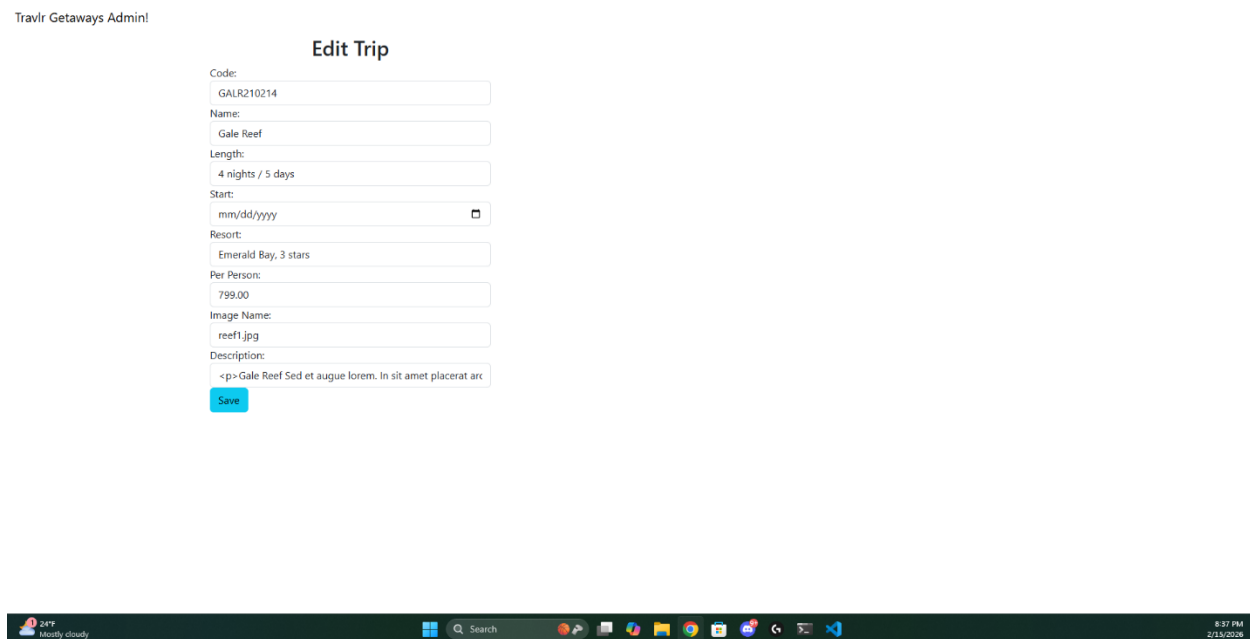
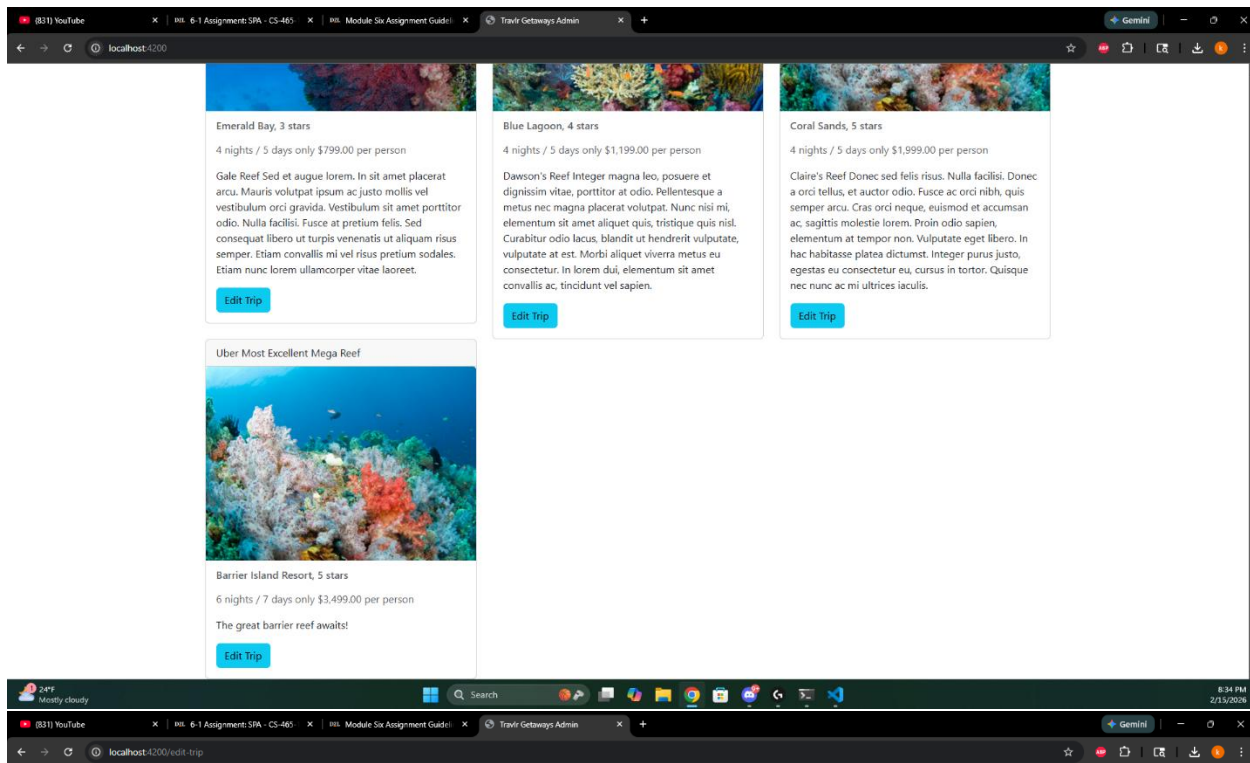
The User Interface



The screenshot shows a web browser window with the URL `localhost:4200/add-trip`. The page title is "Travel Getaways Admin!". The main heading is "Add Trip". Below the heading is a form with the following fields:

- Code:
- Name:
- Length:
- Start: (with a calendar icon)
- Resort:
- Per Person:
- Image Name:
- Description:

At the bottom of the form is a blue "Save" button.



The Angular project structure has many differences from that of the Express HTML customer-facing page. Express sends users to different pages of the website using routes and controllers, which loads a new page when a link is clicked, while Angular is loaded once and updates the page dynamically with the help of components, services, and calls to the API. Express includes folders for routes, controllers, models, and views for navigating pages, requiring a full reload. Angular renders content in the browser using components for user interface, services for API communication, models for the data structure, and routing for client-side navigation.

Some advantages of the SPA functionality are that it is faster and provides a smoother experience than navigating through different HTML pages and has a clear separation of concern between the front-end and back-end, making debugging faster and easier. The downside to SPA is that the initial load of the application will take more time. Additional functionality provided by a SPA compared to simple web application interactions include real-time updates to the page without having to reload after each update to the trips database.

To ensure the SPA is working with the API to GET and PUT data in the database, I used Postman and MongoDB Compass to ensure the database was being updated. Using Postman and MongoDB Compass, I could ensure the status codes were current and that the database was updated correctly. One error that I ran into was a route mismatch, as I had forgotten to have the PUT method call for a tripcode, which was giving me a 404 not found error.